

Project 1: femtoRV32

Milestone 3 Final Report

Pipelined RV32I Processor

Course:	CSCE 3301 – Computer Architecture
Semester:	Spring 2026
Team Member:	Mohamed Azouz – 900225230
Team Member:	Amonios Beshara – 900232679
Instructor:	Dr. Cherif Salama
Submission:	Milestone 3 Final Submission

Contents

1	Introduction	3
2	Final Architecture Overview	3
3	Pipelined Datapath	4
3.1	Stage 0: Instruction Fetch and Decode	4
3.2	Stage 1: Execute and Memory	5
3.3	Stage 2: Writeback	5
4	Single-Port Unified Memory	5
5	Instruction Set Support	6
6	Control Unit	6
7	ALU and ALU Control	7
8	Immediate Generator	7
9	Hazard Handling	8
9.1	Structural Hazards	8
9.2	EX-Stage Forwarding	8
9.3	Load-Use Stall	8
9.4	Control Hazards	8
9.5	ID-Stage Branch Data Hazards	9
10	Bonus Feature 1: ID-Stage Branch Resolution	9
11	Bonus Feature 2: Test Program Generator	10
12	Testing Methodology	11
12.1	Directed Instruction Tests	11
12.2	Representative Simulation Results	12
12.3	Waveform-Based Validation	12
13	Issues Encountered and Solutions	13
13.1	JAL/JALR Link Register Write-Back	13
13.2	Load-Use Hazards	13
13.3	Unified Memory Address Overlap	13
13.4	ID-Stage Branch Hazards	13
14	Current Limitations and Assumptions	14
15	Conclusion	14
A	Submitted File Organization	14
B	Important Verilog Modules	15

C How to Run the Random Test Generator**15**

1. Introduction

This report documents the final Milestone 3 implementation of our `femtoRV32` RISC-V processor. The design evolved from the Milestone 2 single-cycle processor into a pipelined RV32I implementation that uses a single byte-addressable unified memory for both instruction and data accesses. The processor supports the required RV32I user-level instructions, treats the five specified synchronization/system instructions as halting instructions, includes forwarding and stalling for data hazards, and uses an every-other-cycle issuing scheme to avoid structural conflicts on the single memory port.

The final design is implemented in Verilog and targeted for simulation and FPGA testing on the Nexys A7 trainer kit. The processor datapath is organized as a pipeline in which each stage uses two clock phases, denoted C0 and C1. This gives an effective instruction issue rate of one instruction every two clock cycles, matching the project requirement for using a single single-ported memory without instruction/data memory conflicts.

Two bonus features were also implemented. The first bonus moves branch outcome and target address computation to the ID stage and adds the extra forwarding/stall handling needed by this earlier branch resolution. The second bonus is a C++ test program generator that creates random valid RV32I-style test programs and corresponding memory initialization files.

2. Final Architecture Overview

The final implementation is a 32-bit RV32I processor with the following major blocks:

- Program counter and next-PC selection logic.
- Single unified byte-addressable memory.
- IF/ID, ID/EX, EX/MEM, and MEM/WB pipeline registers.
- Control unit for instruction decoding.
- Register file with two read ports and one write port.
- Immediate generator for I, S, B, U, and J-type formats.
- ALU control unit.
- ALU supporting arithmetic, logical, shift, and comparison operations.
- Forwarding unit for EX-stage operand bypassing.
- Inline load-use and ID-branch hazard detection logic.
- Write-back muxing for ALU results, memory data, PC-relative values, and immediates.
- Nexys A7 debug output paths for LEDs and seven-segment display selection.

The top-level module is `riscv`.

```

1 module riscv(
2     input clk,
3     input rst
4 );

```

3. Pipelined Datapath

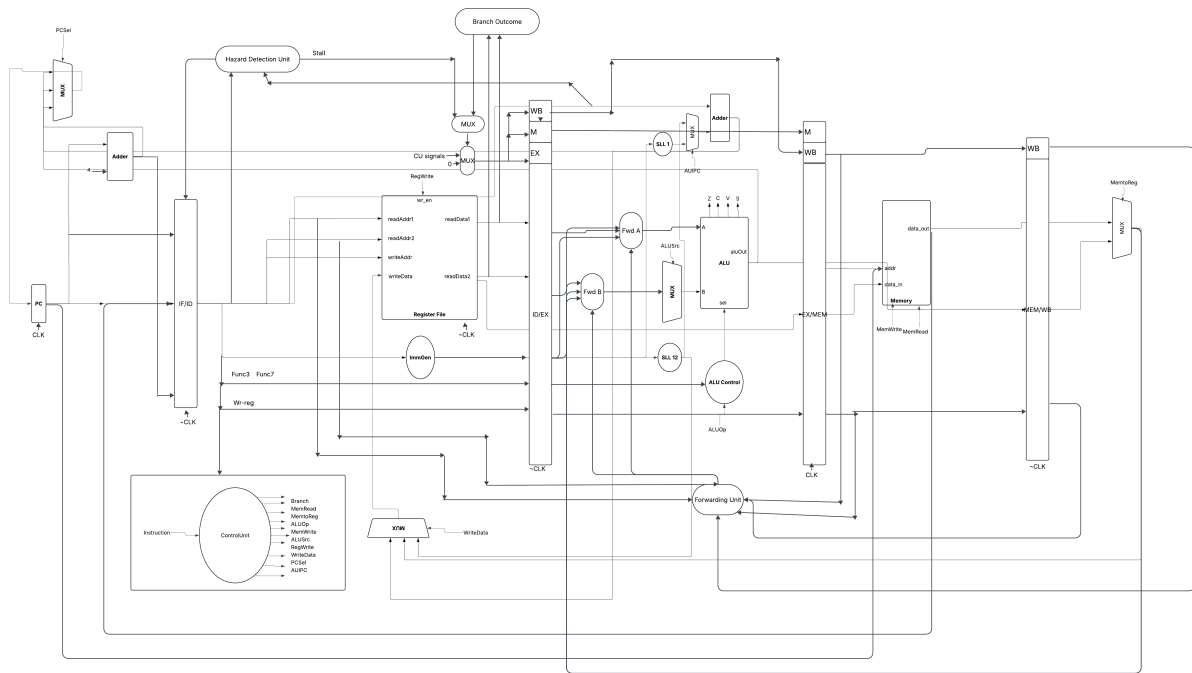


Figure 1: Enter Caption

The design avoids using both positive and negative clock edges. Instead, every sequential element is triggered by the same positive clock edge, and the C0/C1 phase signal determines which pipeline state elements are allowed to load.

3.1. Stage 0: Instruction Fetch and Decode

During C0, the PC selects the instruction address into the unified memory. The returned instruction word is captured into the IF/ID register together with the current PC and PC+4. During C1, the instruction is decoded, the register file is read, the immediate is generated, ID-stage branch decisions are made, and the ID/EX register is loaded.

The IF/ID pipeline register holds:

- IF_ID_PCadd4
- IF_ID_PC
- IF_ID_Instr

3.2. Stage 1: Execute and Memory

During C0, ALU and address generation are performed. The EX stage includes the ALU forwarding muxes, the ALU control unit, the ALU, and branch-add hardware. The results are captured in the EX/MEM pipeline register.

During C1, the unified memory port is used for data memory access if the instruction is a load or store. This is the only time a data memory access is allowed, so it never conflicts with instruction fetch, which occurs on C0.

3.3. Stage 2: Writeback

During C0, the MEM/WB result is written back to the register file if `RegWrite` is asserted. The write-back muxing path selects among:

- PC+4 for JAL/JALR link values.
- `branchAdd` or PC-relative values.
- ALU result or memory load result.
- Immediate value for LUI.

4. Single-Port Unified Memory

Milestone 3 required a single memory for both instructions and data. The final memory module is byte-addressable and exposes only one address/data port to the processor. The top-level datapath selects the memory function based on the C0/C1 phase.

```

1 assign mem_en_single      = c0 ? (~stop_fetch & ~stall_fetch_next)
2                           : (EX_MEM_ctrl[2] | EX_MEM_ctrl[1])
3                           ;
4 assign mem_write_single   = c1 & EX_MEM_ctrl[1];
5 assign mem_funct3_single  = c0 ? 3'b010 : EX_MEM_funct3;
6 assign mem_addr_single    = c0 ? PC_Out[11:0] : EX_MEM_ALUOut
7                           [11:0];
7 assign mem_wdata_single   = EX_MEM_readData_2;

```

This means:

- On C0, the memory address is the PC and the memory output is interpreted as an instruction.
- On C1, the memory address is the EX/MEM ALU result and the memory is used for load/store operations.
- The memory is never asked to fetch an instruction and perform a data access in the same phase.

This approach satisfies the required structural hazard solution: the processor issues an instruction every two clock cycles and schedules IF and MEM accesses on different phases.

5. Instruction Set Support

The final design supports the required RV32I instruction groups listed in Table 1.

Table 1: Required RV32I instruction support

Instruction Group	Instructions
U-type	LUI, AUIPC
Jumps	JAL, JALR
Branches	BEQ, BNE, BLT, BGE, BLTU, BGEU
Loads	LB, LH, LW, LBU, LHU
Stores	SB, SH, SW
I-type ALU	ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI
R-type ALU	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
Halting instructions	ECALL, EBREAK, FENCE, FENCE.TSO, PAUSE

The five special instructions are implemented as halt instructions by asserting `endProgram` in the control unit. This prevents additional program fetches and stops normal PC advancement.

6. Control Unit

The control unit decodes `instruction[6:2]` and produces high-level control signals. The most important control signals are:

- Branch
- MemRead
- MemtoReg
- ALUOp
- MemWrite
- ALUSrc
- RegWrite
- PCSel
- AUIPCSel
- endProgram
- writeData_Sel

The control unit handles `SYSTEM` and `MISC-MEM` opcodes as halting instructions for this project:

```
1 'OPCODE_SYSTEM: begin
2     endProgram = 1'b1;
3     RegWrite   = 1'b0;
4     MemRead    = 1'b0;
5     MemWrite   = 1'b0;
6     Branch     = 1'b0;
7 end
8
9 OPCODE_MISC_MEM: begin
10     endProgram = 1'b1;
11     RegWrite   = 1'b0;
12     MemRead    = 1'b0;
13     MemWrite   = 1'b0;
14     Branch     = 1'b0;
15 end
```

7. ALU and ALU Control

The ALU supports the RV32I arithmetic, logical, shift, and comparison operations. Signed comparisons are implemented using `$signed`, while unsigned comparisons use the natural unsigned interpretation of the 32-bit vectors.

The ALU control unit uses `ALUOp`, `funct3`, and the appropriate `funct7` bit to distinguish operations such as:

- ADD vs. SUB
- SRL vs. SRA
- SRLI vs. SRAI

This separation reduces complexity in the main control unit and keeps detailed ALU instruction decoding inside `ALUControlUnit`.

8. Immediate Generator

The immediate generator supports all immediate formats required by RV32I:

- I-type immediates for loads, immediate ALU instructions, and JALR.
- S-type immediates for stores.
- B-type immediates for branches.
- U-type immediates for LUI and AUIPC.
- J-type immediates for JAL.

The generated immediate is passed through the ID/EX pipeline register and is also used directly in the ID stage for the branch-resolution bonus.

9. Hazard Handling

Correct hazard handling was one of the central requirements of Milestone 3. The final design handles structural, data, load-use, and control hazards.

9.1. Structural Hazards

The single unified memory would create a conflict if instruction fetch and data access happened in the same cycle. The design avoids this by using the C0/C1 phase schedule:

- C0: instruction fetch.
- C1: load/store memory access.

Therefore, the single memory port is used by only one operation in each phase.

9.2. EX-Stage Forwarding

The forwarding unit compares the source registers in ID/EX with destination registers in EX/MEM and MEM/WB. It generates two 2-bit select signals:

```

1 if (EX_MEM_RegWrite && (EX_MEM_RegisterRd != 5'b00000) &&
2   (EX_MEM_RegisterRd == ID_EX_RegisterRs1))
3   forwardA = 2'b10;
4 else if (MEM_WB_RegWrite && (MEM_WB_RegisterRd != 5'b00000) &&
5   (MEM_WB_RegisterRd == ID_EX_RegisterRs1))
6   forwardA = 2'b01;
```

The same logic is applied for `forwardB`. This supports ALU-to-ALU and MEM/WB-to-ALU dependencies.

9.3. Load-Use Stall

A load-use hazard occurs when an instruction immediately after a load needs the loaded register. The value is not ready soon enough for normal EX-stage forwarding. The final design detects this condition inline inside `riscv.v`:

```

1 assign load_use_hazard =
2   ID_EX_ctrl[5] && (ID_EX_rd != 5'd0) &&
3   ((if_id_uses_rs1 && (ID_EX_rd == IF_ID_Instr[19:15])) ||
4   (if_id_uses_rs2 && (ID_EX_rd == IF_ID_Instr[24:20])));
```

When this hazard occurs:

- PC is stalled.
- IF/ID is stalled.
- A bubble is inserted into ID/EX by zeroing the control signals.

9.4. Control Hazards

Branches and jumps redirect the PC. In the final design, branch and jump outcomes are resolved in the ID stage as a bonus feature. Wrong-path instructions are prevented from committing by inserting bubbles/flushes when a redirect is detected.

9.5. ID-Stage Branch Data Hazards

Moving branch resolution to ID creates new hazards because the branch comparator may need values that are still in EX/MEM or MEM/WB. The final design solves this by forwarding into the ID-stage comparator and JALR target calculation:

```

1 assign id_rs1_value =
2     (ex_mem_forward_valid && (EX_MEM_rd == IF_ID_Instr[19:15])) ?
3     ex_mem_forward_value :
4     (mem_wb_forward_valid && (MEM_WB_rd == IF_ID_Instr[19:15])) ?
5     writeData :
6     readData_1;

```

A special branch-load hazard is also detected when a branch or JALR depends on a value that is currently being loaded:

```

1 assign id_branch_load_hazard =
2     (if_id_is_branch || if_id_is_jalr) &&
3     EX_MEM_ctrl[2] &&
4     (EX_MEM_rd != 5'd0) &&
5     ((EX_MEM_rd == IF_ID_Instr[19:15]) ||
6     (if_id_is_branch && (EX_MEM_rd == IF_ID_Instr[24:20])));

```

10. Bonus Feature 1: ID-Stage Branch Resolution

The first implemented bonus moves branch outcome and target-address computation from the EX stage to the ID stage. This reduces the branch penalty because the processor decides whether to redirect the PC earlier.

The ID-stage branch decision uses forwarded operand values:

```

1 always @(*) begin
2     branchRes = 1'b0;
3     if (!id_stall_hazard) begin
4         if (if_id_is_jal || if_id_is_jalr) begin
5             branchRes = 1'b1;
6         end else if (if_id_is_branch && Branch) begin
7             case (IF_ID_Instr[14:12])
8                 'BR_BEQ: branchRes = (id_rs1_value ==
9                     id_rs2_value);
10                'BR_BNE: branchRes = (id_rs1_value !=
11                    id_rs2_value);
12                'BR_BLT: branchRes = ($signed(id_rs1_value) <
13                    $signed(id_rs2_value));
14                'BR_BGE: branchRes = ($signed(id_rs1_value) >=
15                    $signed(id_rs2_value));
16                'BR_BLTU: branchRes = (id_rs1_value <
17                    id_rs2_value);
18                'BR_BGEU: branchRes = (id_rs1_value >=
19                    id_rs2_value);
20                default: branchRes = 1'b0;
21            endcase
22        end
23    end
24 end

```

```

17     end
18 end

```

The branch and JAL target use:

```

1 assign id_branch_target = IF_ID_PC + immOut;

```

The JALR target uses:

```

1 assign id_jalr_sum      = id_rs1_value + immOut;
2 assign id_jalr_target = {id_jalr_sum[31:1], 1'b0};

```

This implements the required JALR target masking by clearing bit 0.

11. Bonus Feature 2: Test Program Generator

The second implemented bonus is a C++ test generator. The generator produces random but valid instruction sequences, writes them to `program.hex`, creates randomized data memory contents, and writes a human-readable `trace.txt`. It uses a data memory base address of 2048 and reserves 512 data words.

The generator includes instruction groups for:

- R-type arithmetic/logical instructions.
- I-type ALU instructions.
- Shift-immediate instructions.
- U-type instructions.
- Branch instructions.
- Load instructions `lb`, `lh`, `lw`, `lbu`, `lhu`.
- Store instructions `sb`, `sh`, `sw`.
- Jumps and system/fence encoders.

The generator tracks known register values so it can compute expected results for generated instructions. It also models byte and halfword memory behavior for load/store tests.

Example output files:

- `program.hex`: byte-wise program initialization file.
- `data.hex`: byte-wise data initialization file.
- `trace.txt`: readable instruction trace with PC and machine-code words.

The random program ends with an infinite `jal x0, 0` loop so that the processor remains in a stable state after the generated program completes.

12. Testing Methodology

The final testing strategy combines directed simulation tests with the random test program generator.

12.1. Directed Instruction Tests

The directed tests are organized by instruction class. Table 2 summarizes the planned and executed categories.

Table 2: Directed test suite

Test ID	Coverage	Expected Evidence
1	LUI, AUIPC	Register results match expected immediate and PC-relative values
2	JAL, JALR	Link registers receive PC+4; wrong-path instructions are flushed
3	BEQ, BNE, BLT, BGE, BLTU, BGEU	Correct branch taken/not-taken behavior
4	SB, SH, SW, LB, LH, LW, LBU, LHU	Correct byte/half-word/word storage, sign extension, and zero extension
5	I-type ALU instructions	Correct immediate ALU and shift-immediate results
6	R-type ALU instructions	Correct register-register ALU results
7	ECALL	PC stops updating
8	EBREAK	PC stops updating
9	FENCE	PC stops updating
10	FENCE.TSO	PC stops updating

Test ID	Coverage	Expected Evidence
11	PAUSE	PC stops updating
12	Hazard tests	ALU forwarding, store-data forwarding, load-use stall, and branch-related hazards

12.2. Representative Simulation Results

During simulation, the JAL/JALR test passed with the following representative output:

```
=== Test 2: JAL/JALR ===
PASS: x1 = 0x00000004
PASS: x2 = 0x00000018
PASS: x3 = 0x0000001c
PASS: x5 = 0x00000000
PASS: x6 = 0x00000000
TEST 2 PASSED
```

The hazard test also passed after the inline load-use stall and ID-branch hazard logic were added:

```
=== Test 12: Hazards ===
PASS: x1 = 0x0000000a
PASS: x2 = 0x00000014
PASS: x3 = 0x0000001e
PASS: mem[0xa0] = 0x00000055
PASS: x5 = 0x00000055
PASS: x6 = 0x000000aa
PASS: x8 = 0x00000000
PASS: x9 = 0x0000007b
TEST 12 PASSED
```

These outputs verify the most important pipeline features: link write-back, jump redirection, EX-stage forwarding, store-data forwarding, load-use stalling, and branch hazard behavior.

12.3. Waveform-Based Validation

In Vivado/XSim, the following signals were monitored:

- PC_Out
- phase

- IF_ID_Instr
- ID_EX_ctrl
- EX_MEM_ctrl
- MEM_WB_ctrl
- forwardA, forwardB
- load_use_hazard
- id_branch_load_hazard
- stall_fetch_next
- branchRes
- writeData
- selected register-file values

13. Issues Encountered and Solutions

13.1. JAL/JALR Link Register Write-Back

An early issue caused JAL and JALR to jump correctly but fail to write PC+4 into the destination register. The cause was that the control signals for the jump instruction itself were being flushed when `branchRes` was asserted. The solution was to avoid clearing the EX/MEM control bits for the jump instruction and only flush wrong-path younger instructions.

13.2. Load-Use Hazards

The hazard test originally produced an incorrect result where an ADD following a load used the load address instead of the loaded value. Inline load-use detection was added so the dependent instruction is held in IF/ID and a bubble is inserted into ID/EX. After this change, the hazard test produced the correct value.

13.3. Unified Memory Address Overlap

Because instruction and data share one memory, test data placed too close to the program can overwrite or overlap with instruction bytes. The tests were adjusted to place data in a separate region, starting from address 2048, while instruction code remains at the lower memory addresses.

13.4. ID-Stage Branch Hazards

Moving branch resolution to the ID stage created new operand timing problems. Branches and JALR may need operands that have not yet reached the register file. The solution was to add ID-stage forwarding from EX/MEM and MEM/WB and to stall on branch-load hazards.

14. Current Limitations and Assumptions

The final design assumes that:

- Programs and data are placed in non-overlapping regions of the unified memory.
- The directed tests are the primary evidence for all 42 required instructions.
- The random generator is used as stress testing in addition to directed coverage.
- FPGA implementation was not tested in time, so it was removed from the project.

The C++ random generator includes encoders and instruction categories for the required instruction classes, but directed tests remain necessary for guaranteed coverage of halt instructions and edge-case control-flow behavior.

15. Conclusion

Milestone 3 completes the transition from the Milestone 2 single-cycle datapath to a pipelined RV32I processor that uses a single byte-addressable unified memory. The final datapath uses C0/C1 phase timing to avoid structural hazards, includes pipeline registers between all major stages, and implements forwarding and stalling for data hazards.

The processor supports the required RV32I instruction groups and treats `ECALL`, `EBREAK`, `FENCE`, `FENCE.TSO`, and `PAUSE` as halting instructions. Directed simulation tests validate jumps, hazards, loads/stores, ALU operations, branches, and halt behavior. The first bonus feature moves branch resolution into the ID stage with added data-hazard support, and the second bonus feature provides a C++ random test program generator.

The design is prepared for Nexys A7 implementation using the provided constraints and debug output ports. Final FPGA submission evidence should include waveform screenshots and board photos showing representative test cases running on hardware.

A. Submitted File Organization

The final submission directory should include:

- `Verilog/`: all Verilog source files.
- `test/`: Examples of generated `program.hex`, `data.hex`, and traces.
- `Test Generator/`: `testGen.cpp` and a few generated sample programs.
- `journal/`: one journal file per team member.
- `readme.txt`: assumptions, known issues, and instructions for simulation/FPGA testing.
- `report.pdf`: final version of this report.

B. Important Verilog Modules

Module	Purpose
<code>riscv</code>	Top-level pipelined CPU integration, C0/C1 phase control, hazard logic, branch resolution, and debug outputs
<code>unified_memory</code>	Single-port byte-addressable unified instruction/data memory
<code>ControlUnit</code>	Main instruction decoder and high-level control signal generator
<code>ALUControlUnit</code>	ALU operation decoder using <code>ALUOp</code> , <code>funct3</code> , and <code>funct7</code>
<code>ALU</code>	Arithmetic, logical, shift, and comparison datapath
<code>reg_file</code>	32-entry integer register file
<code>rv32_ImmGen</code>	Immediate extraction and sign extension
<code>ForwardingUnit</code>	EX-stage ALU operand forwarding control
<code>nbit_reg</code>	Parameterized register used for PC and pipeline registers
<code>nbit2x1mux</code> , <code>nbit_4x1mux</code>	Datapath muxes used for operand, PC, and write-back selection
<code>RCA</code> , <code>FA</code>	Ripple-carry adder and full-adder building blocks

C. How to Run the Random Test Generator

Compile and run the generator:

```

1 g++ testGen.cpp -o testGen
2 ./testGen <number_of_instructions>

```

This produces:

- `program.hex`
- `data.hex`
- `trace.txt`

These files can be loaded into the unified memory model by changing the filepath and used to test the processor using the simulation output.